

Separating “effort” from “verification”

Proposals for visible reliability in generative AI assistants, arising from a user-model conversation

Date	26 September 2026
Model	Claude, configured identifier claude-opus-5-5 (the model serving any given turn may differ)
Surface	Claude (Cowork), web browser; effort setting shown to the user: Medium (default)
User	Iñaki Gauna, R&D engineer (industrial NDT / AI systems)
Authorship	Ideas and critical review by the user; drafting and structuring by the model, at the user’s request
Status	Proposal for discussion. Not an evaluated design.

1. Summary

In a single conversation, the model produced several fluent statements that were unsupported or inconsistent with what it had said earlier. The user caught each one; the model recognised each error immediately once it was pointed out. What was missing was a review step, not knowledge. From this the conversation produced three proposals:

- **A. A “Verify claims” control, independent of the effort level** (Off / Auto / Always). Reasoning longer is not the same as checking claims.
- **B. Per-claim status labels** (verified, corrected, not verified, not verifiable), so users can see where to apply their own scrutiny.
- **C. An expert evaluation channel** where domain experts assess unaltered model output blind, and their detections are compared with an automatic evaluator to expose the model’s shared blind spots.

The common thread: today the burden of detecting confabulation falls on the user, who usually cannot tell a confabulated answer from a correct one because both arrive with the same speed and fluency. These proposals aim to make failures *visible and predictable*, not to promise that they disappear.

2. Evidence from the session

The following statements were produced by the model and challenged by the user. Quotes are translated from Spanish.

Statement (model)	Problem	Could the model have avoided it?
“...your reports about it come from a system whose consciousness nobody disputes.”	Absolute quantifier that is false (eliminativism, illusionism and other-minds scepticism do dispute it).	Yes. The relevant positions were known to the model; it cited them itself in the next reply.
“Your own experience raises no doubts for you.”	Unfounded claim about the user’s mental states; also philosophically contested (e.g. Schwitzgebel).	Yes. No evidence was available about the user.
“Some hallucinations appear when that brake misfires.”	In the previous turn the model had argued that “confabulation” was the better term. It adopted the term of the source it was citing.	Yes. The conflicting commitment was in the conversation context.

Statement (model)	Problem	Could the model have avoided it?
"If I could identify them, I would have corrected them."	Overclaim. The session itself showed errors the model could identify but did not correct.	Yes. Counter-examples were in the same conversation.

Caveat: a model's readiness to "correct" itself when challenged is not, on its own, evidence that it knew the right answer, because models tend to defer to users. In these cases the correction rests on arguments the model had already made, and the model also rejected part of one of the user's premises.

Working taxonomy (hypothesis, not diagnosis)

The model proposed four causes of uncorrected errors. They are the model's own reconstruction of its behaviour and should be treated as hypotheses.

Cause	Description	Addressable by review?
1. Not knowing	The error is in the model's knowledge (e.g. a widely repeated misattributed quote).	Only with external sources
2. Knowledge not applied	The model knows enough but does not check the output against it.	Yes
3. Unexpressed doubt	Internal uncertainty is not reflected in the confident tone of the text.	Yes, via calibration
4. Consistency pressure	Once generated, text conditions what follows, which favours defending it over revising it; the opposite failure is deferring to the user.	Partly

Illustration of cause 1: the model could write "As Einstein said, insanity is doing the same thing over and over and expecting different results." A self-review by the same model would likely pass it too, because the misattribution is common in text. Quote Investigator found no evidence that Einstein said it; the earliest known instance is from 1981, in a newspaper report quoting an AI-Anon member. Only an external source catches this kind of error.

3. Proposal A: a "Verify claims" control independent of effort

Today's effort selector (Low to Max) implies that more effort yields more reliable answers. Longer reasoning may include some self-review, but that review is neither systematic nor visible, and longer reasoning can produce more elaborate confabulations. Effort and verification are separate dimensions and should be controllable separately.

State	Behaviour
Off	Current behaviour.
Auto (recommended default)	Verification scales with risk. It is triggered by claim types that commonly fail: citations and attributions, figures and dates, absolute quantifiers ("nobody", "always"), claims about the user, and claims about the model itself.
Always	Every checkable claim is verified before the answer is shown.

Design requirements

- **External checks, not only re-reading.** Use search, code execution, and confirmation that a cited source actually supports the claim. Self-review by the same model shares the generator's blind spots; Huang et al. (2024) found that self-correction without external feedback often fails to improve reasoning and sometimes degrades it.

- **Verify before display.** Draft, split into atomic claims, check high-risk claims in a fresh context (without the reasoning that produced the draft), then show.
- **Show the outcome** (see Proposal B). An invisible check is a black box.
- **Show the cost**, as the Max option already does (for example “~2× usage”).
- **Careful naming.** Avoid “Self-verification” or anything that suggests guaranteed correctness. “Verify claims” promises less and describes the function better.
- **Keep a record of commitments made in the conversation.** Terms, decisions and positions fixed earlier are checked against each new answer. This targets errors like the terminology switch above.

Minimum viable version: even a simple first version is valuable, provided it marks what it could *not* verify. Without that, a weak verifier could create false confidence and reduce trust instead of adding it.

4. Proposal B: per-claim status labels

Each paragraph or conclusion carries a small, distinctive label, expandable to per-claim detail.

Label	Meaning
Verified	Checked against an external source; link provided so the user can check it directly.
Corrected	Evidence contradicted the draft; the claim was changed before display.
Not verified	Checkable, but could not be checked. This means “not checked”, not “false”.
Not verifiable	Opinion, inference or proposal. Labelled so it does not look like a failure.

Design notes

- **Label everything.** The implied truth effect (Pennycook et al., 2020): when warnings are attached to only some false headlines, the unlabelled ones are perceived as more accurate. Unlabelled claims would be read as verified.
- **“Not verified” must not read as “false”.** Wording and visual design should make clear it marks something still to check.
- **Granularity.** Paragraph-level labels are readable but mix claims of different status; paragraph labels that expand to claim level are a reasonable compromise.
- **Label fatigue.** If almost everything is “not verified”, users stop looking. Verification must be effective where checking is possible.

5. Proposal C: an expert evaluation channel

A dedicated mode for registered users with domain expertise, possibly with incentives such as usage credits. The aim is to measure how detectable the model’s real errors are for experts in each domain, and where the model and its own evaluator share blind spots.

Evolution of the idea

The first version proposed that the model deliberately inject errors for experts to find. Two problems led to its revision:

- **Distribution mismatch:** planted errors may differ from natural confabulations, a known limitation of defect seeding in software testing.
- **Dual use:** optimising a model to produce errors experts miss trains convincing deception.

Revised design

- An **unaltered generator instance** (equivalent to the production model) produces the domain content. The errors are therefore natural, and no one trains deception.
- A **separate evaluator** analyses and scores the text before anything is shown, in a fresh context and ideally with external tools. A model from a different family can be added for comparison.
- The **expert evaluates blind**: they mark what they consider wrong *before* seeing the evaluator's scores, to avoid anchoring and automation bias. The two assessments are compared afterwards.
- **Disagreements are adjudicated** against a source, a test or a second expert, because experts can be wrong too.
- **Targeted sampling**: focus on question types where the model is known to fail more often (citations, specific parameters, standards), to collect enough cases.

Detected by	Interpretation
Evaluator and expert	Error the system can already catch.
Evaluator only	Possible model contribution, or false positive. Needs adjudication.
Expert only	Blind spot shared by generator and evaluator. The most valuable signal.
Neither	Unknown. Surfaces only through later source checks.

The channel is also a test bench for Proposal A: the evaluator is the verifier, and the experts measure whether it works.

How it reaches the general public

Most users will never use this channel. Its value reaches them in two ways. (1) **Product**: the blind spots it finds improve the verifier, which shows up as labels in everyone's conversations. (2) **Transparency**: publishing per-domain metrics (for example, the share of expert-detected errors the verifier missed) makes the commitment verifiable. Such metrics may lower naive trust in the short term, but they build *calibrated* trust (Lee & See, 2004). "Evaluated by experts" must not be used as a blanket guarantee.

6. Note for technical and industrial domains

In domains such as industrial ultrasonic inspection (the user's field), many questions concern API behaviour, device command sets, signal-processing parameters or inspection standards. There:

- Web search is a weak verifier, because specialised content is scarce online and "not found" would be the norm. Useful sources are code execution and unit tests, manufacturer SDKs and manuals, the applicable standard, and the user's own codebase. The verification control should let users say *what to verify against*.
- The most useful label for an engineer is "not verified": it shows exactly where to look.
- In safety-relevant work (e.g. aerospace NDT), verification should guide human review, not replace it. Responsibility stays with the engineer and the qualified inspector.

7. Risks and open problems

Risk	Mitigation
Evaluator shares the generator's blind spots	External tools; a model from another family; expert channel to measure it
Automation bias: users trust "verified" too much	Show sources; label everything; wording that avoids guarantees
Cost and latency	Auto mode scaled to risk; visible cost indicator
Over-abstention makes the assistant useless	The goal is calibration, not silence
External sources can be wrong	Show the source so the user can judge it; adjudication in the expert channel
Error type 1 (not knowing) cannot be eliminated	Reduced only where there is something external to check against
Dual use (expert channel)	No deliberate error injection; data used to improve detectors and calibration, not generators

8. Prior art

The ideas are not new individually. A web search on 26 September 2026 found:

Work	Relation to these proposals
Google Gemini, "Double-check response"	Closest product precedent. On request, after generation, checks the response with Google Search and highlights statements (green: similar content found; orange: different or not found). It is manual and after the fact, does not correct, has no risk-based mode, and "similar content" does not mean the source supports the claim. Availability in 2026 not fully confirmed.
Chain-of-Verification; RARR; SAFE; FActScore; ClaimCheck	Techniques for splitting output into claims and verifying or revising them, some with search. ClaimCheck shows real-time checking with small models, which lowers cost.
CriticGPT (McAleese et al., 2024)	People deliberately inserted subtle bugs into model-written code to evaluate model critics against human reviewers. Closest precedent for Proposal C, with roles reversed.
Defect seeding (Weinberg 1971; Mills 1972)	Planting known defects to estimate how many a review process misses.
Scalable oversight (Bowman et al., 2022); LLM-as-a-judge (Zheng et al., 2023)	Related research on combining AI and human evaluation, and on how reliable model judges are compared with humans.
Why Language Models Hallucinate (Kalai et al., 2025)	Argues that training and evaluation reward guessing over acknowledging uncertainty. Supports making abstention and "not verified" first-class outcomes.

What appears less explored (not found, which is not proof of absence): a persistent verification setting that is *independent of effort*, with a *risk-based automatic mode*, verification *before display*, labels on every claim including unverified ones, and a blind expert channel designed to measure shared blind spots.

9. Why it matters

From the user's side, the prevailing perception is that capability is already sufficient for many use cases, but reliability is not. The problem is less the average error rate than its unpredictability: a human expert fails in predictable ways, whereas a model's excellent and fabricated answers look the same. In the study by Dell'Acqua et al. (2023), 758 BCG consultants using AI did better on tasks inside the model's capability frontier (12.2% more tasks, 25.1% faster, over 40% higher quality), but on a task outside it they were 19 percentage points less likely to reach a correct solution than those working without AI. The proposals in this document do not require a perfect model. They require that its failures be visible.

10. Epistemic status of this document

- **Well established:** the errors in Section 2 are verbatim from the session (translated). The prior-art entries and references were checked by web search on 26 September 2026.
- **Inferred:** the four-cause taxonomy, the relative value of the "expert only" category, the recommended default (Auto), and the expected effects on trust. None has been evaluated with users.
- **Still to verify:** current availability of Gemini's double-check feature; whether Weinberg (1971) or Mills (1972) should be credited as the origin of defect seeding, checked against primary sources; what the product's effort levels do internally, which the model does not know.

References

- Bowman, S. R., et al. (2022). Measuring Progress on Scalable Oversight for Large Language Models. arXiv:2211.03540.
- Dell'Acqua, F., McFowland III, E., Mollick, E. R., et al. (2023). Navigating the Jagged Technological Frontier. Harvard Business School Working Paper 24-013. SSRN 4573321.
- Dhuliawala, S., et al. (2024). Chain-of-Verification Reduces Hallucination in Large Language Models. Findings of ACL 2024 (arXiv:2309.11495).
- Frankish, K. (2016). Illusionism as a Theory of Consciousness. *Journal of Consciousness Studies*, 23(11-12), 11-39.
- Gao, L., et al. (2023). RARR: Researching and Revising What Language Models Say, Using Language Models. ACL 2023.
- Google. View related sources & double-check responses from Gemini Apps. support.google.com/gemini/answer/14143489.
- Hicks, M. T., Humphries, J., & Slater, J. (2024). ChatGPT is bullshit. *Ethics and Information Technology*, 26, 38. doi:10.1007/s10676-024-09775-5.
- Huang, J., et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR 2024 (arXiv:2310.01798).
- Johnson, M. K., Hashtroudi, S., & Lindsay, D. S. (1993). Source monitoring. *Psychological Bulletin*, 114(1), 3-28.
- Kadavath, S., et al. (2022). Language Models (Mostly) Know What They Know. arXiv:2207.05221.
- Kahneman, D., & Klein, G. (2009). Conditions for intuitive expertise: A failure to disagree. *American Psychologist*, 64(6), 515-526.
- Kalai, A. T., Nachum, O., Vempala, S. S., & Zhang, E. (2025). Why Language Models Hallucinate. arXiv:2509.04664.
- Lee, J. D., & See, K. A. (2004). Trust in Automation: Designing for Appropriate Reliance. *Human Factors*, 46(1), 50-80.
- Lindsey, J., et al. (2025). On the Biology of a Large Language Model. Transformer Circuits Thread.
- Lindsey, J. (2025). Emergent Introspective Awareness in Large Language Models. Transformer Circuits Thread; arXiv:2601.01828.
- McAleese, N., et al. (2024). LLM Critics Help Catch LLM Bugs. arXiv:2407.00215.

- Mills, H. D. (1972). On the Statistical Validation of Computer Programs. IBM Federal Systems Division.
- Min, S., et al. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. EMNLP 2023.
- Pennycook, G., Bear, A., Collins, E. T., & Rand, D. G. (2020). The Implied Truth Effect. *Management Science*, 66(11), 4944–4957.
- Putta, A. R., Devasier, J., & Li, C. (2025). ClaimCheck: Real-Time Fact-Checking with Small Language Models. arXiv:2510.01226.
- Quote Investigator (2017). Insanity Is Doing the Same Thing Over and Over Again and Expecting Different Results. quoteinvestigator.com/2017/03/23/same/.
- Schwitzgebel, E. (2011). *Perplexities of Consciousness*. MIT Press.
- Weinberg, G. M. (1971). *The Psychology of Computer Programming*. Van Nostrand Reinhold.
- Wei, J., et al. (2024). Long-form factuality in large language models. NeurIPS 2024 (arXiv:2403.18802).
- West, P., et al. (2024). The Generative AI Paradox: “What It Can Create, It May Not Understand”. ICLR 2024 (arXiv:2311.00059).
- Zheng, L., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks.